



Angular – Framework JS

Créer une application web interactive / Maîtriser les concepts clés d'Angular /
Utiliser les bonnes pratiques à son avantage



Thomas Ferreira – Développeur Full Stack

Diplômé de l'ESGI spécialité Architecture des Logiciels en 2021, je travaille chez Wizards Technologies depuis octobre 2019.

J'ai participé à plusieurs projets depuis mes débuts chez WT :

- Du Wordpress :vomi:
- Du VueJS
- De l'Angular 2+
- Beaucoup de Typescript

Je bosse majoritairement sur des frameworks front-end mais aussi sur du back-end occasionnellement.

J'interviens également sur des missions de Business Development, de Lead Développeur ou de Formation et je donne des cours (HTML/CSS, Conception BDD, Algo, Architecture Logiciel, Angular, ...).

Contact : thomas.tfdv+cours@gmail.com



Organisation du cours

Fondamentaux d'Angular - Navigation et Communication - Formulaire
et Validation - Outils d'Angular (Directives et Pipes) - Tests et
Déploiements - Bonnes pratiques



Qu'allons-nous voir sur **Angular** ?

- **Fondamentaux :**

- Introduction à Angular 2
- Environnement de Développement
- Composants et Modules
- Template et Directives intégrés

- **Navigation et Communication :**

- Routage
- Services et Injection de Dépendance
- HTTP et Communication

- **Formulaires et Validation :**

- Formulaires "Template Driven"
- Formulaires "Reactive Form"

- **Directives et Pipes :**

- Directives Personnalisées
- Pipes Intégrées et Personnalisées

- **Tests, Déploiement et Bonnes pratiques :**

- Tests Unitaires et d'Intégration
- Déploiement
- Bonnes pratiques



Fondamentaux d'Angular

Introduction à Angular 2 - Environnement de Développement -
Composants et Modules - Template et Directives intégrés



Introduction à Angular 2

Angular est un **framework** de développement Web populaire soutenu par Google.

Il est conçu pour créer des **applications Web dynamiques**, riches et robustes.

Angular favorise l'utilisation de bonnes pratiques de programmation et **offre des outils puissants** pour la gestion des composants, du routage, des formulaires, des services, et plus encore.

Malgré la concurrence avec React et Vue.js, Angular reste l'un des frameworks les plus populaires pour le développement Web.





Différences AngularJS et Angular 2+ et 17+

AngularJS (v1.x) été publié en 2010 mais une nouvelle version appelé Angular (v2+) est sortie en 2016. Angular 2 marque une **réécriture complète** d'AngularJS, avec des changements significatifs dans la syntaxe et l'architecture dans le but de résoudre les limitations d'angular JS.

La liaison bidirectionnelle a été simplifiée, la performance et la modularité a été améliorée, et **TypeScript** est devenu le langage de choix.

Depuis la version 14 et définitivement à la 17, Angular passe à l'utilisation de composants **Standalone** afin de **simplifier l'architecture** qui ajoutait une couche de complexité et parfois de la duplication inutile.



```
1 //AngularJS
2 // $scope.message = 'Hello World!';
3 //Angular2+
4 message: string = 'Hello World!';
5
```



Architecture d'une app **Angular**

- Modules
- Composants
- Services
- Directives
- Templates
- Pipes

```
1 @Component({  
2   ··selector: ··'app-root',  
3   ··template: ··'<h1>{{title}}</h1>',  
4 })  
5 export class AppComponent {  
6   ··title: string = 'My Angular App'  
7 }  
8
```



Installation d'Angular

Angular CLI, l'outil de ligne de commande pour Angular, dépend de Node.js (^16.14.0 || ^18.10.0) et npm, donc il est essentiel de les installer en premier.

Le processus d'installation est simple et bien documenté sur les [sites officiels](#).



```
1 node --version
2 npm install -g @angular/cli@16
3 ng version
```

```
Angular CLI: 16.1.7
Node: 18.17.0
Package Manager: npm 9.6.7
OS: darwin arm64
```

```
Angular:
...
```

Package	Version
@angular-devkit/architect	0.1601.7 (cli-only)
@angular-devkit/core	16.1.7 (cli-only)
@angular-devkit/schematics	16.1.7 (cli-only)
@schematics/angular	16.1.7 (cli-only)



Création d'un projet avec Angular CLI

Angular CLI est un outil puissant qui simplifie la création, le développement, et le test d'applications Angular. Avec une simple commande, `ng new mon-app`, vous pouvez générer une nouvelle application Angular avec une structure bien organisée, des outils de test préconfigurés, et bien plus encore.

```
> ng new angular-example
? Would you like to add Angular routing? Yes
? Which stylesheet format would you like to use? SCSS [ https://sass-lang.com/documentation/syntax#scss ]
CREATE angular-example/README.md (1069 bytes)
CREATE angular-example/.editorconfig (274 bytes)
CREATE angular-example/.gitignore (548 bytes)
CREATE angular-example/angular.json (2919 bytes)
CREATE angular-example/package.json (1048 bytes)
CREATE angular-example/tsconfig.json (901 bytes)
CREATE angular-example/tsconfig.app.json (263 bytes)
CREATE angular-example/tsconfig.spec.json (273 bytes)
CREATE angular-example/.vscode/extensions.json (130 bytes)
CREATE angular-example/.vscode/launch.json (470 bytes)
CREATE angular-example/.vscode/tasks.json (938 bytes)
CREATE angular-example/src/main.ts (214 bytes)
CREATE angular-example/src/favicon.ico (948 bytes)
CREATE angular-example/src/index.html (300 bytes)
CREATE angular-example/src/styles.scss (80 bytes)
CREATE angular-example/src/app/app-routing.module.ts (245 bytes)
CREATE angular-example/src/app/app.module.ts (393 bytes)
CREATE angular-example/src/app/app.component.scss (0 bytes)
CREATE angular-example/src/app/app.component.html (22709 bytes)
CREATE angular-example/src/app/app.component.spec.ts (1018 bytes)
CREATE angular-example/src/app/app.component.ts (220 bytes)
CREATE angular-example/src/assets/.gitkeep (0 bytes)
Installing packages (npm)...
```

```
> .vscode
> node_modules library root
v src
  v app
    app.component.html
    app.component.scss
    app.component.spec.ts
    app.component.ts
    app.module.ts
    app-routing.module.ts
  v assets
    .gitkeep
    favicon.ico
    index.html
    main.ts
    styles.scss
    .editorconfig
    .gitignore
    angular.json
    package.json
    package-lock.json
    README.md
    tsconfig.app.json
    tsconfig.json
    tsconfig.spec.json
```

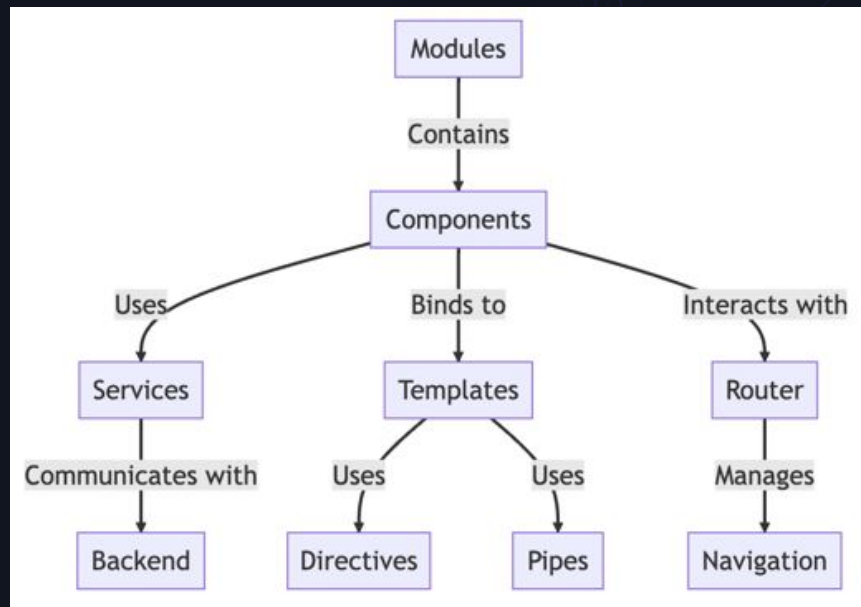


Structure d'un projet **Angular**

Un projet Angular a une **structure spécifique**, avec des dossiers pour les composants, les services, les modules, et autres.

Le fichier `angular.json` contient les paramètres de **configuration** du projet.

Comprendre cette structure vous aidera à naviguer et organiser votre code efficacement.





Les composants d'Angular

Les composants sont **au cœur** d'Angular.

Chaque composant est **une classe** TypeScript qui gère une partie de la page.

Un composant peut posséder un template HTML et un style CSS.

Vous pouvez créer des composants en utilisant Angular CLI avec **la commande** `ng generate component nom-du-composant`.

```
> ng g c new-component
CREATE src/app/new-component/new-component.component.scss (0 bytes)
CREATE src/app/new-component/new-component.component.html (28 bytes)
CREATE src/app/new-component/new-component.component.spec.ts (602 bytes)
CREATE src/app/new-component/new-component.component.ts (230 bytes)
UPDATE src/app/app.module.ts (501 bytes)
```

Pourquoi faire des composants ?

- Réutilisabilité
- Maintenance
- Performances
- Lisibilité



Les modules d'Angular

Les modules **regroupent** des éléments liés, comme des composants, des directives, et des services.

Les modules jouent un **rôle crucial** dans la structuration, l'organisation et la scalabilité des applications Angular.

Chaque application Angular a au moins un **module racine**, appelé `AppModule`.

Pourquoi faire des modules ?

- Réutilisabilité
- Maintenance
- Performances
- Lisibilité

```
1 @NgModule({  
2   declarations: [HelloComponent],  
3   imports: [BrowserModule],  
4   bootstrap: [AppComponent],  
5 })  
6 export class AppModule {}  
7
```



Les templates et interpolations

Un template désigne la partie de l'interface utilisateur d'un composant.

Contrairement au HTML statique ordinaire, un template Angular est **dynamique**, ce qui signifie qu'il peut changer en fonction des données et de la logique définies dans le fichier TypeScript du composant.

L'**interpolation** est une syntaxe qui permet d'insérer dynamiquement des valeurs issues du composant

```
1 @Component({  
2   selector: 'app-title',  
3   template: '<h1>{{title}}</h1>',  
4 })  
5 export class TitleComponent {  
6   title = 'Bienvenue dans Angular!'  
7 }  
8
```



Les boucles et conditions

Les directives sont des **instructions** dans les templates qui disent à Angular comment rendre le DOM.

Les **directives structurelles**, comme `*ngFor`` et `*ngIf``, modifient la structure du DOM, tandis que les directives d'attribut, comme `ngClass`` et `ngStyle``, changent l'**apparence** ou le **comportement** d'un élément.

En utilisant `*ngIf``, l'élément ne sera **pas afficher`** `display: none`` si la variable qui est vérifiée renvoie `false``.
`*ngFor`` lui va **dupliquer l'élément`** sur lequel il se trouve autant de fois qu'il y a d'itération dans la variable donnée.



```
1 <div *ngIf="showMessage">
2   · Welcome to Angular!
3 </div>
4 <ul>
5   · <li *ngFor="let item of items">{{item}}</li>
6 </ul>
```



Exercice : Fondamentaux

Créer une application avec :

- Au minimum 4 composants.
- Des boucles dans le template
- Des conditions dans le template

Exercice fil conducteur :

- Création d'un Quiz sur la thématique que vous souhaitez
 - Composant principale, header et footer
 - Composant pour la home page
- Rendre les questions et réponses dynamiques
- Ajouter un calcul de score



Les Events bindings

Les Events Bindings avec Angular sont le fait de **lier des actions à des évènements** afin de déclencher du code.

Par exemple (click)="..." permet d'**exécuter du code** au click de l'utilisateur sur un élément HTML.

Il en existe d'autres :

- (keyup),
- (keydown)
- (mouseover)
- (mouseout)
- (submit)



```
1 export class MyComponent {  
2   ..onButtonClick() {  
3     ...console.log('clicked');  
4   }  
5 }
```



```
1 <button (click)="onButtonClick()">Click Me</button>  
2
```



Router – Services – HTTP

Routing - Navigation - Services - Injection de dépendance -
HTTP et communication



Utilisation du Router

Angular permet principalement de créer des SPA (Single Page Application).

C'est de cette manière que l'on a l'impression qu'il n'y a pas de changement de page mais tout se passe du côté du Router qui va permettre d'avoir plusieurs pages.

Le Router vous permet de définir des routes vers lesquels naviguer.

```
1 import { RouterModule, Routes } from '@angular/router';
2 import { HomeComponent } from '../home/home.component';
3 import { AboutComponent } from '../about/about.component';
4
5 const appRoutes: Routes = [
6   { path: 'home', component: HomeComponent },
7   { path: 'about', component: AboutComponent },
8 ];
9
10 @NgModule({
11   imports: [RouterModule.forRoot(appRoutes)],
12   exports: [RouterModule],
13 });
14 export class AppRoutingModule {}
15
```



Navigation de route

La navigation se fait de 2 manières principales :

- À l'aide du service **Router** qui permet de naviguer via le composant d'une route à l'autre.
- À l'aide de la **directive** `routerLink` dans le template sur une balise `<a>`.

```
1 import { Component } from  
  '@angular/core';  
2 import { Router } from '@angular/router';  
3  
4 @Component({  
5   selector: 'app-navigate-button',  
6   template: '<button  
  (click)="navigateToAbout()">Aller à la  
  page A propos</button>',  
7 })  
8 export class NavigateButtonComponent {  
9   constructor(private router: Router) {}  
10  
11   navigateToAbout() {  
12     this.router.navigate(['/about']);  
13   }  
14 }  
15
```

```
1 <a routerLink="/about">A propos</a>  
2
```



Paramètres de route

Vous pouvez également ajouter des **paramètres** à vos routes.

```
1 import { RouterModule, Routes } from '@angular/router'
2 import { HomeComponent } from '../home/home.component'
3 import { UserComponent } from '../user/user.component'
4
5 const appRoutes: Routes = [
6   { path: 'home', component: HomeComponent },
7   { path: 'user/:id', component: UserComponent },
8 ]
9
10 @NgModule({
11   imports: [RouterModule.forRoot(appRoutes)],
12   exports: [RouterModule],
13 })
14 export class AppRoutingModule {}
15
```

```
1 // user-list.component.ts
2 export class UserListComponent {
3   users = [
4     { id: 1, name: 'Alice' },
5     { id: 2, name: 'Bob' }
6   ];
7
8   constructor(private router: Router) {}
9
10   viewDetails(user) {
11     this.router.navigate(['/user', user.id]);
12   }
13 }
```



Récupérer des paramètre de route

Vous pouvez récupérer les paramètres passés dans vos routes avec le service ActivatedRoute.

```
1 // user-list.component.ts
2 export class UserListComponent {
3   users = [
4     { id: 1, name: 'Alice' },
5     { id: 2, name: 'Bob' }
6   ];
7
8   constructor(private router: Router) {}
9
10  viewDetails(user) {
11    this.router.navigate(['/user', user.id]);
12  }
13 }
```

```
1 import { Component, OnInit } from '@angular/core';
2 import { ActivatedRoute } from '@angular/router';
3
4 @Component({
5   selector: 'app-user-details',
6   template: '<h2>Details for User ID: {{userId}}</h2>'
7 })
8 export class UserDetailsComponent implements OnInit {
9   userId: number;
10
11   constructor(private route: ActivatedRoute) {}
12
13   ngOnInit() {
14     this.userId = +this.route.snapshot.paramMap.get('id');
15   }
16 }
17
```



Les services

- Classe avec un objectif défini
- Partager des données / logic métier / interaction avec des API
- Garder les composants légers et déléguer les tâches

```
1 @Injectable({  
2   ··providedIn: 'root'  
3 })  
4 export class UserService {  
5   ··private users = [{id: 1, name: 'Alice'},  
6     {id: 2, name: 'Bob'}];  
7   ··getUsers() {  
8     ···return this.users;  
9   }  
10 }
```



L'injection de dépendance

L'injection de dépendance est un mécanisme de conception qui permet à Angular d'**injecter des dépendances** dans des composants ou des services plutôt que de les créer en interne.

Pour qu'un service soit **injectable**, il doit être fourni à l'injection de dépendance. En utilisant `providedIn: 'root'`, vous fournissez le service au **niveau racine** de l'application.

Pour utiliser un service dans un composant, vous devez l'injecter dans le **constructeur** du composant. Il en va de même pour l'utilisation dans d'autres services.

```
1 @Component({  
2   selector: 'app-user-list',  
3   template: '<div *ngFor="let user of users">{{  
4     user.name }}</div>'  
5 })  
6 export class UserListComponent {  
7   users = [];  
8   constructor(private userService: UserService) {  
9     this.users = userService.getUsers();  
10  }  
11 }
```



HTTP et Communication Backend

Angular possède son propre module pour effectuer des appels HTTP, le module `HttpClient`.

Avec les `méthodes` nécessaires à l'utilisation d'une API REST, GET, POST, PUT, etc.

Attention, ces méthodes vont retourner des `observables` auxquelles il va falloir souscrire.

```
1 import { HttpClient } from '@angular/common/http';  
2  
3 @Injectable({  
4   providedIn: 'root',  
5 })  
6 export class UserService {  
7  
8   constructor(private http: HttpClient) {}  
9  
10  getUsers() {  
11    return this.http.get('https://randomuser.me/api/');  
12  }  
13  
14  getUserById(id: string) {  
15    return this.http.get(`https://api.example.com/users/${id}`);  
16  }  
17  
18  createUser(user: any) {  
19    return this.http.post('https://api.example.com/users', user);  
20  }  
21 }  
22
```



Exercice : Router – Service – HTTP

1. Passer tout le contenu du composant Home dans un nouveau service
 - Utiliser ce service dans le composant pour garder le bon fonctionnement du quiz
2. Ajouter un composant Quiz qui reprend ce que vous avez fait
 - De sorte à épurer le contenu du composant Home
3. Créer un bouton sur la home page pour naviguer vers le quiz
 - Ajouter une route pour afficher le composant quiz
4. Créer un JSON-SERVER
 - Ajouter vos questions et réponses
5. Consommer l'API pour récupérer et afficher vos questions réponses



Composants – Formulaire

Cycle de vie - Communication - Formulaire - Validation



Cycle de vie des composants

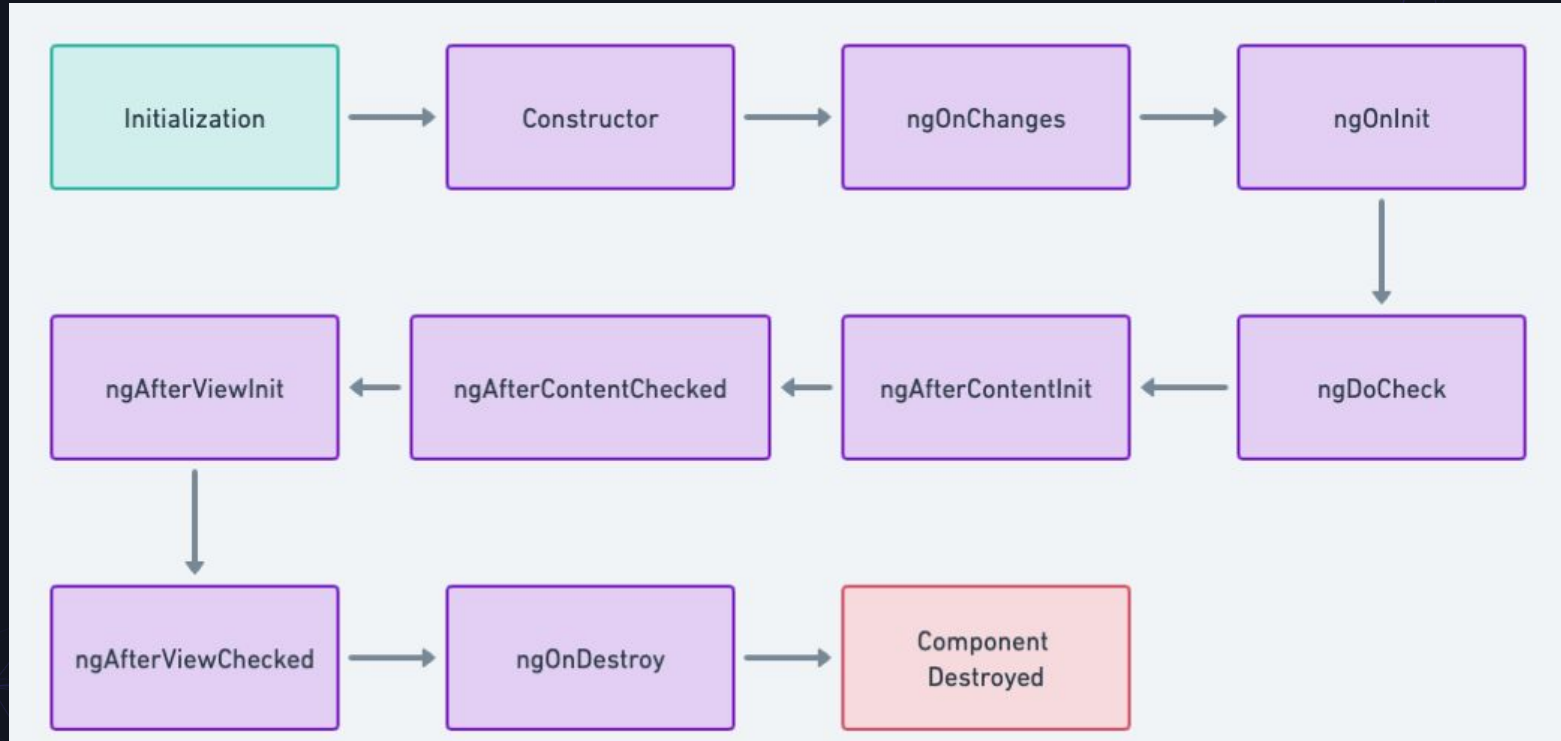
Chaque composant Angular a un **cycle de vie**, différentes étapes qu'il parcourt de sa création à sa destruction. Angular fournit des hooks pour intervenir à ces différentes étapes :

- **ngOnInit** : Exécuté après que les @Input ont été vérifiées.
- **ngOnChanges** : Exécuté à chaque fois qu'un @Input change.
- **ngDoCheck** : Détecte et agit sur les changements que Angular ne peut pas détecter seul.
- **ngOnDestroy** : Juste avant qu'Angular détruise le composant.

```
1 export class MonComposant implements OnInit, /*...*/, OnDestroy {  
2   ngOnInit() {  
3     console.log("Le composant est initialisé.");  
4   }  
5   ngOnChanges() {  
6     console.log("Un @Input a changé.");  
7   }  
8   ngDoCheck() {  
9     console.log("Le composant a été modifié.");  
10  }  
11  ngAfterContentInit() {  
12    console.log("Le contenu du composant a été initialisé.");  
13  }  
14  ngAfterContentChecked() {  
15    console.log("Le contenu du composant a été modifié.");  
16  }  
17  ngAfterViewInit() {  
18    console.log("La vue du composant a été initialisée.");  
19  }  
20  ngAfterViewChecked() {  
21    console.log("La vue du composant a été modifiée.");  
22  }  
23  ngOnDestroy() {  
24    console.log("Le composant va être détruit.");  
25  }  
26 }
```



Cycle de vie des composants





Communication entre composants

Les composants peuvent **communiquer** entre eux, en particulier le parent avec l'enfant et vice-versa :

- **Input** : Permet de passer une valeur du composant parent au composant enfant.
- **Output** : Permet au composant enfant d'émettre un événement vers le parent.

```
1 // enfant.component.ts
2 @Component({
3   selector: 'app-enfant',
4   template: `<button (click)="envoyerEvent()">Cliquez
    moi</button>`
5 })
6 export class EnfantComponent {
7   @Input() nom: string;
8   @Output() monEvent = new EventEmitter<string>();
9   ...
10  envoyerEvent() {
11    this.monEvent.emit('Un événement de l'enfant!');
12  }
13 }
```

```
1 // parent.component.ts
2 @Component({
3   selector: 'app-parent',
4   template: `
5     <app-enfant [nom]="nomPourEnfant"
6       (monEvent)="traiterEvent($event)"></app-enfant>
7   `
8 })
9 export class ParentComponent {
10   nomPourEnfant = 'Mon nom';
11   ...
12   traiterEvent(event: string) {
13     console.log(event);
14   }
15 }
```



Formulaire : Template Driven

Les formulaires de type “**Template Driven**” utilisent le `FormsModule` et les directives associées pour créer et gérer les formulaires directement dans le template.

```
1 // app.module.ts
2 import { FormsModule } from '@angular/forms';
3
4 @NgModule({
5   imports: [FormsModule]
6 })
```

```
1 // monform.component.ts
2 @Component({
3   selector: 'app-monform',
4   template: `
5     <form #f="ngForm" (ngSubmit)="soumettre(f)">
6       <input [(ngModel)]="username" name="username"
7         required>
8       <button type="submit">Soumettre</button>
9     </form>
10  `
11 })
12 export class MonFormComponent {
13   username: string;
14   soumettre(form: NgForm) {
15     console.log(form.value);
16   }
17 }
```



Formulaire : Reactive Form

Les formulaires de type “Reactive Form” nécessitent l'importation de `ReactiveFormsModule` et `FormsModule` et utilisent une approche plus robuste, flexible et réactive pour créer des formulaires. Son utilisation se fait à l'aide de `FormGroups` et `FormControls`.

Les `FormControls` représentent une entrée individuelle - un **élément de formulaire** et possèdent des propriétés et des méthodes pour gérer la valeur et la validation de l'entrée.

```
1 // monform.component.ts
2 import { FormControl } from
  '@angular/forms';
3
4 @Component({
5   selector: 'app-monform',
6   template: `
7     <input [formControl]="username">
8   `
9 })
10 export class MonFormComponent {
11   username = new FormControl('');
12
13   getUsername() {
14     console.log(this.username.value);
15   }
16
17   setUsername() {
18     this.username.setValue('Mon nom');
19   }
20 }
```



Formulaire : Reactive Form

Les `FormGroups` représentent un groupe d'entrées - un formulaire. Il est donc constitué de plusieurs `FormControls`.

Ils possèdent également des propriétés et des méthodes pour gérer les valeurs et la validation du groupe.

Les `FormGroups` sont ce que l'on recommande pour des formulaires complexes sur lesquels on souhaite avoir du contrôle groupé.

```
1 // monform.component.ts
2 import { FormGroup, FormControl } from
  '@angular/forms';
3 @Component({
4   selector: 'app-monform',
5   template: `
6     <form [formGroup]="profileForm">
7       <input formControlName="firstName"
6       type="text">
8       <input formControlName="lastName"
6       type="text">
9     </form>
10   `
11 })
12 export class MonFormComponent {
13   profileForm = new FormGroup({
14     firstName: new FormControl(''),
15     lastName: new FormControl(''),
16   });
17 }
```



Formulaire : Validators

Angular fournit des **validateurs** courants pour vous aider à valider les entrées utilisateur dans les formulaires.

```
1 import { FormControl, Validators } from '@angular/forms';  
2 /*...*/  
3 const control = new FormControl('', Validators.required);
```

```
1 import { AbstractControl, ValidationErrors } from '@angular/forms';  
2 /*...*/  
3 function forbiddenNameValidator(control: AbstractControl): ValidationErrors | null {  
4   const forbidden = /.admin/.test(control.value);  
5   return forbidden ? { 'forbiddenName': { value: control.value } } : null;  
6 }
```



Exercice : Composants – Formulaire

1. Ajouter des 'inputs'

- Créer un composant Question et Answer avec Angular CLI
- Utiliser le composant Answer dans le composant Question en lui passant les réponses

2. Ajouter un formulaire en 'Template Driven'

- Ajouter un champs pour que l'utilisateur puisse entrer son nom
- Ajouter son nom en paramètre de l'url quiz pour l'afficher dans le quiz

3. Ajouter un formulaire en 'Reactive Forms'

- Ajouter la possibilité de créer un compte et de s'identifier à l'aide d'un formulaire en 'Reactive Forms'

4. Ajouter des validations

- Ajouter des validations sur chacun des champs d'inscription



Pipe – Directive – Composant

Pipes (intégrées et personnalisées) / Directives personnalisées /
Composants dynamiques



Pipes intégrées

Les Pipes en Angular sont des fonctions qui prennent en entrée une valeur et la transforment en une sortie désirée. C'est donc notamment très utile pour **re-formater** des données.

Angular propose un ensemble de pipes intégrées pour gérer les tâches courantes de transformation :

- **date** : Pour formater les dates.
- **uppercase** et **lowercase** : Pour transformer le texte en majuscules ou minuscules.
- **currency** : Pour formater un nombre en tant que devise monétaire.
- **percent** : Pour transformer un nombre en pourcentage.
- **json** : Pour afficher un objet sous forme de chaîne JSON.



```
1 <p>{{ currentDate | date:'fullDate' }}</p>  
2 <p>{{ title | uppercase }}</p>  
3 <p>{{ price | currency:'USD':true }}</p>
```



Pipes personnalisées

Il est également possible de créer ses propres pipes pour des cas d'utilisation spécifiques.

Par exemple pour créer une pipe pour abrégé une chaîne de caractères après un certain nombre de caractères.

```
1 import { Pipe, PipeTransform } from '@angular/core'
2
3 @Pipe({
4   name: 'abbreviate',
5 })
6 export class AbbreviatePipe implements PipeTransform {
7   transform(value: string, limit = 10): string {
8     if (value.length > limit) {
9       return `${value.substr(0, limit)}...`
10    }
11    return value
12  }
13 }
14
```



Directives personnalisées

Les directives permettent d'ajouter un comportement à un élément DOM existant ou à un composant.

Contrairement aux composants, elles ne possèdent pas de template propre.

Par exemple on pourrait créer une directive qui change la couleur de fond d'un élément lorsqu'il est survolé.

```
1 import { Directive, Renderer2, ElementRef, HostListener } from '@angular/core';  
2 -  
3 @Directive({  
4   selector: '[appHoverHighlight]'  
5 })  
6 export class HoverHighlightDirective {  
7   constructor(private el: ElementRef, private renderer: Renderer2) {}  
8 -  
9   @HostListener('mouseenter') onMouseEnter() {  
10     this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'yellow');  
11   }  
12 -  
13   @HostListener('mouseleave') onMouseLeave() {  
14     this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'transparent');  
15   }  
16 }
```



Pipes ou Directives ?

	Pipe	Directive
Objectif principal	transformer des données pour l'affichage	étendre ou modifier le comportement d'un élément DOM ou d'un composant.
Réactivité	statiques ou réactives basées sur des valeurs changeantes, mais généralement pas à des événements utilisateur.	généralement en réponse à des événements tels que des interactions utilisateur (clics, survols, etc).
Utilisation dans les templates	<code>{{ value myPipe }}</code>	<code><div appMyDirective></div></code>
Cas d'utilisation typique	Transformer ou formater une donnée avant de l'afficher à l'utilisateur.	Interagir ou modifier le comportement d'un élément ou d'un composant en fonction d'une condition ou d'une interaction utilisateur.